Revisiting exact-penalty linear programming

A projection-path walker for the regime where Newton slows down

Jeffery Kline

Version 0.1.0-draft
17 August 2026

Living repository: `https://github.com/jeff-kline/exact-penalty-lp-walker`.
Permanent version archive: not yet assigned; this draft has no DOI.

Abstract

In 2002, Mangasarian built a Newton LP method from a quadratic penalty that becomes exact at a finite parameter. In our tests it works well when constraints outnumber variables by roughly 5 to 10, but slows sharply below about 2 or 3 — a regime that includes all of Netlib-27, and the one the **t-walker** is built for.

At penalty parameter t, projecting tb onto the dual feasible set D gives the multiplier $y(t) = P_D(tb)$. We claim no new path or homotopy: in 1997, Pinar implemented a predictor-corrector solver for an equivalent penalty program on this path. The **t-walker** follows $y(t)$ face by face, finding breakpoints by a minimum-ratio test. Our t-walker contributes a sparse breakpoint implementation that carries face coefficients, support statuses, and a rank-revealing core across pivots; repairs rank-deficient faces; and accepts only primal-dual pairs passing a KKT certificate on the original unscaled data. We report measured KKT errors.

The walker beats staged Newton in eleven of twelve certified low-ratio cells. The fastest certified of two walkers and Newton is a median 2.82 times faster than Newton at ratios from 1.1 to 2 and cuts Netlib-27 time from 11.60 to 9.60 seconds. For both, simplex crossover cuts median Netlib error by about three orders. Choosing the fastest of three methods per problem is hindsight rather than a dispatcher, and the result still trails HiGHS by 16.6 times on Netlib-27.

Status. Public prototype, not a peer-reviewed paper. The accompanying repository holds the code, fixtures, and frozen records behind every measurement reported here. This release is in draft state: no stable tag, no permanent archive, and no DOI.

1 The gap

Mangasarian showed that a quadratic penalty program yields an exact LP solution at a sufficiently large but finite penalty parameter [Mangasarian and Meyer, 1979, Mangasarian, 1984, 2004]. He used that result to build a Newton method for LPs. Empirically, Newton excels when constraints outnumber variables by roughly 5 to 10 and often reaches the LP solution in only a few nonlinear iterations. But a gap remains. Newton slows sharply below roughly 2 to 3 constraints per variable, a regime that includes every model in the Netlib benchmark panel used here and arises in practical linear programs. This note addresses that gap. Separately, because solver status strings do not measure accuracy on a common scale, we re-evaluate every method under one original-data certificate rather than trusting its own stopping test; that is a measurement protocol, not a claim that the walker is the more accurate method.

We built a complementary method, the **t-walker**, for this regime. The walker follows the exact dual projection path from face to face instead of solving the penalty problem afresh at a sequence of penalty values. On the tested low-ratio problems, t-walker outpaces Newton in eleven of the twelve measured cells; the exception is $m = 200$ at ratio 1.1, where Newton is 1.24 times faster and the triangular seed recovers the cell. Choosing the fastest certified result among the default-seed walker, the triangular-seeded walker, and staged Newton improves on staged Newton alone across Netlib. The envelope still trails HiGHS by more than an order of magnitude.

We test both raw methods against the same original-data accuracy standard, and both pass on the problems they solve. A common simplex crossover cuts their median Netlib error by about three orders of magnitude. After crossover, their median errors have the same order as HiGHS IPM with crossover, although the synthetic tails remain mixed and t-walker still has three Netlib DNFs.

On the current 24-cell synthetic panel (25, 50, and 200 variables; ratios 1.1 through 32; five interleaved runs), the fastest certified result among the three methods runs a median **2.82 times faster than Newton alone for ratios at most 2**, with cell speedups from 1.69 to 11.76. Across all 24 cells, the measured lower envelope chooses the default-seed walker 11 times, the triangular-seeded walker 6 times, and Newton 7 times. This measured frontier uses hindsight; it does not yet provide an automatic dispatcher.

The standard Netlib-27 panel gives a harder test. The current frontier takes **9.60 s**, versus **11.60 s** for shipped Newton and **0.370 s** for the faster HiGHS engine on each model. Its geometric-mean ratio to HiGHS is **16.6 times**, down from Newton's 26.5 but still far from production parity. That is progress, not victory. HiGHS draws on roughly eighty years of simplex and interior-point ideas and implementation practice [Huangfu and Hall, 2018]; we assembled the walker prototype in a handful of AI-assisted engineering sessions.

2 Two computational strategies

Write the primal and dual as

$$\min_x d^\mathsf{T} x \quad \text{s.t.} \quad Bx \geq b, \qquad \max_y b^\mathsf{T} y \quad \text{s.t.} \quad B^\mathsf{T} y = d,\ y \geq 0.$$

Assume the dual feasible polyhedron $D = \{y \geq 0 : B^\mathsf{T} y = d\}$ is nonempty. Mangasarian's penalty program minimizes $d^\mathsf{T} x + \frac{t}{2}\|(b - Bx)_+\|^2$; its multiplier $y = t(b - Bx(t))_+$ is exactly

$$y(t) = P_D(tb) = \arg\min_{y \in D} \frac{1}{2}\|y - tb\|^2.$$

The optimizer path is continuous and piecewise affine. Write

$$\phi(t) = \max_{y \in D} \left(t\, b^\mathsf{T} y - \tfrac{1}{2}\|y\|^2 \right),$$

the value of the projection program up to the constant $\frac{1}{2}t^2\|b\|^2$. Then ϕ is piecewise quadratic with $\phi'(t) = b^\mathsf{T} y(t)$ piecewise affine, and the path direction determines its curvature. The walker therefore follows an affine state on each face even though ϕ is quadratic there. Mangasarian's penalty program has value $\phi(t)/t$, which is not itself piecewise quadratic.

3 How the walker advances

On a face with positive dual support W, the projection equations reduce to

$$y_W(t) = tg + h.$$

Method	What it maintains	Why it wins	Where it pays
Staged Newton	A fixed-t semismooth solve, with staged increases in t and terminal polishing.	Crosses many support changes in a few iterations; excellent on tall synthetic systems.	Expensive identification and factor work near the low-ratio regime.
t-walker	One affine face segment $y(t) = tg + h$ and the next ratio-test event.	Cheap, explicit progress near the square end; checks every accepted endpoint.	Pays for every breakpoint; long paths dominate tall and difficult models.
Triangular-seeded t-walker	The same walk, but a QR/triangular fixed-$t = 0$ projection seed.	Can make initialization and the first face cheaper.	Presently requires full column rank and fails closed on some cells.
HiGHS simplex/IPM	Mature production bases, pricing, presolve, scaling, and crossover.	Robust reference performance and coverage.	A different engineering scale; used as the benchmark, not as our contribution.

Here $g = (I - \Pi_W)b_W$, where Π_W is the orthogonal projector onto the range of B_W, and $h = B_W(B_W^\mathsf{T} B_W)^+ d$. Each active coordinate $y_i(t)$ must remain nonnegative. Each inactive coordinate has an affine multiplier margin that must also remain nonnegative. The walker finds the next breakpoint with a minimum-ratio test over these margins. At a tie, it exchanges rows at fixed t to resolve degeneracy; those exchanges do not count as progress. A valid next segment must reach a strictly larger t or prove that the path is terminal.

Within a face the curvature of ϕ is $\phi''(t) = g^\mathsf{T} g$, so the test $g^\mathsf{T} g \leq$ tolerance detects a stationary face. A stationary face is necessary but not sufficient for termination: g vanishes identically on any face whose active rows are independent, which includes every vertex of D the path passes through on its way to the optimum. The walker therefore accepts an endpoint only when the audited ratio scan additionally shows no forward event and the original-data certificate passes. Treating $g^\mathsf{T} g \leq$ tolerance alone as a stopping rule would return a suboptimal point on any instance whose path dwells at a vertex.

We made the implementation faster and more stable by treating the walker state like a numerical simplex state. The solver now carries the face coefficients, sparse products, support statuses, and a rank-revealing square core across pivots instead of solving unrelated least-squares problems at each one. A one-row change updates that state. The solver fully reconstructs it only after a failed error bound, a rank change, or a rare repair. We made six main changes:

1. native C++ face algebra and cached affine artifacts instead of default library calls at every pivot;

2. a stable square orthogonal core, with a small weak-subspace SVD/COD repair when the face is rank deficient;

3. deterministic statuses for dependent zero rows, so fixed-t exchanges cannot cycle merely by revisiting an equivalent support label;

4. iterative refinement and forward-horizon error bounds that the walker invokes only when coefficient uncertainty can change the next event;

5. fixed-t seed and endpoint repairs that feed the unchanged original-data acceptance gate; and

6. a final primal-dual certificate, independent of t, on the unscaled input.

The prototype remains hybrid. By default, an in-process Newton method builds the walker's $t = 0$ seed. An optional triangular/Wolfe-space seed instead factors B and solves the same fixed-t projection without forming a dense null-space basis. On difficult Netlib faces, the walker can also invoke HiGHS-backed endpoint selection or terminal face repair. We count these calls toward walker time, and every result must pass the original-data certificate. The benchmark therefore does not yet isolate wholly native path algebra.

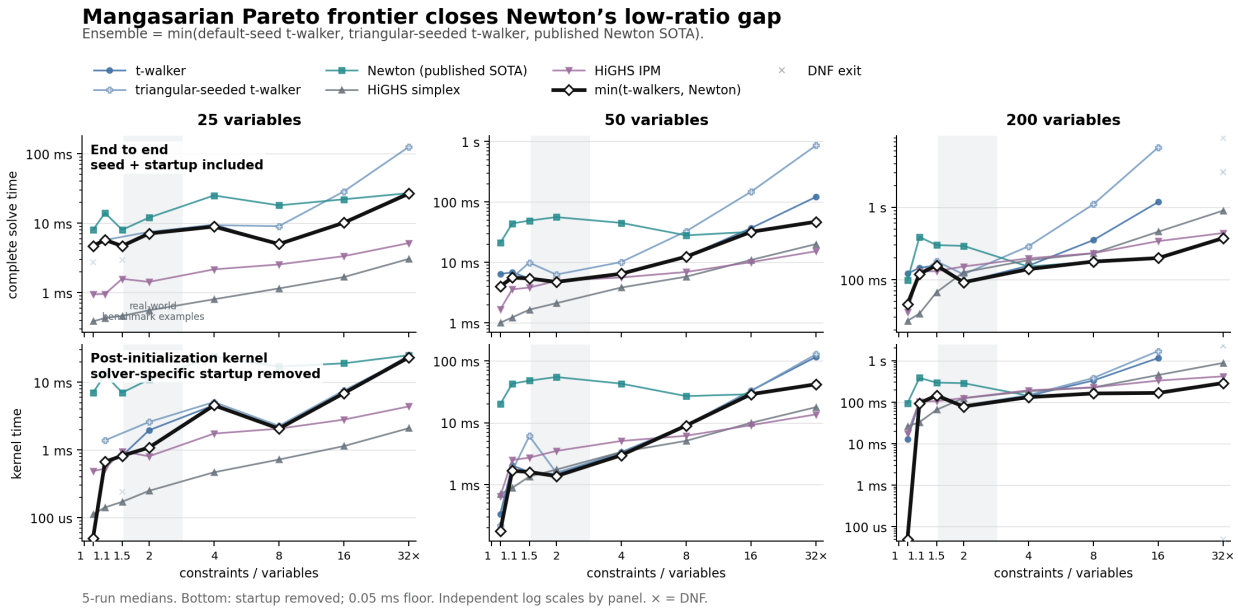

Figure 1: Complete time above and post-initialization kernel time below. The black line shows the fastest certified default-seed walker, triangular-seeded walker, or Newton result. The measured axis starts at 1.1; we retain 1.0 only as a reference. The generator changes aspect ratio and planted support geometry together, so the plot establishes a useful regime, not a theorem that aspect ratio alone is causal.

4 Accuracy, coverage, and what remains

Solver status strings do not measure accuracy on a common scale. We therefore reevaluated every available primal-dual pair on the original inequality form with one original-data KKT certificate, using componentwise scaling on the primal and dual residuals. The score is the largest of four normalized errors: primal infeasibility, dual equality, violation of $y \geq 0$, and the primal-dual gap. The acceptance tolerance is 10^{-7}. Exact feasibility with a zero gap implies exact optimality, so a passing score is a genuine certificate; the first two terms alone would be componentwise backward errors, but the nonnegativity and gap terms are not, so we do not call the composite a backward error. DNF and resource-limit cases count as coverage failures, not zero-error samples.

For t-walker and Newton, we send each solver's dual point to the same warm-started HiGHS simplex postprocessor and then apply the same original-data certificate. We do not pass t-walker's retained basis. This procedure gives us a portable common crossover, not the proposed native basis route.

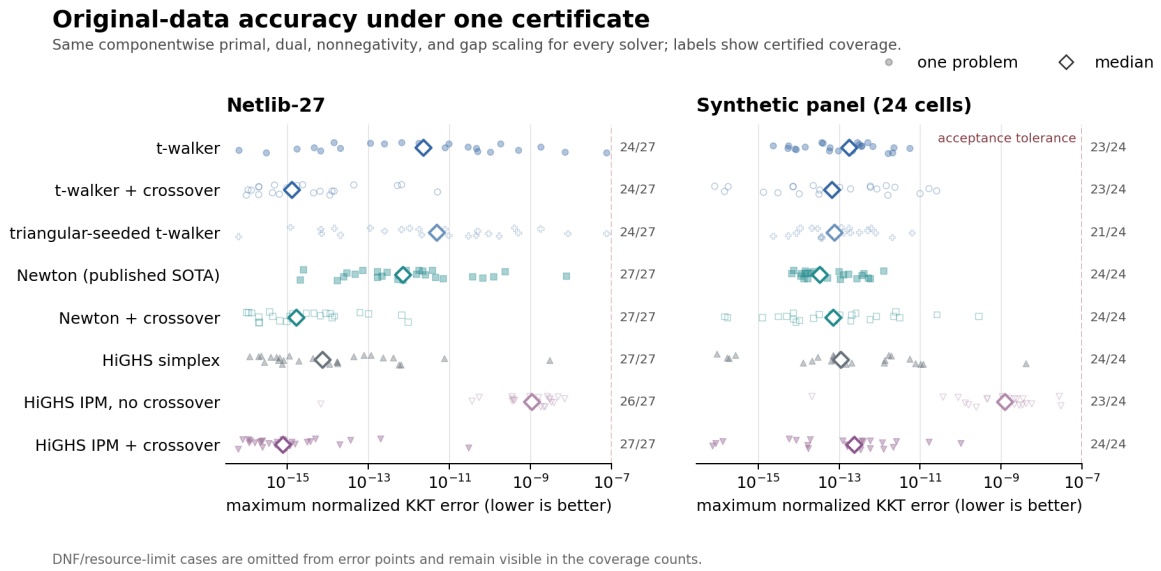

Figure 2: Each dot represents one problem under the same original-data certificate; diamonds mark medians, and the labels at right show coverage. Common crossover cuts the Netlib median error by about three orders for both methods. The synthetic results are mixed: t-walker's median improves but its tail widens, while Newton's median and tail worsen but remain inside tolerance. HiGHS IPM crossover restores one failed certificate in each suite. Crossover cannot repair a DNF because a DNF supplies no terminal point.

These results do not show that a new LP solver beats HiGHS. They show that Newton and the walker cover different regimes. On the controlled family, Newton has a reproducible weak band that the walker fills. On Netlib, the measured Pareto frontier saves about 2.0 seconds over Newton alone. The portfolio still trails mature solvers by more than an order of magnitude and does not certify every model.

Three problems remain. First, turn the hindsight frontier into a dispatcher that uses observed geometry—rank, weak singular values, and identification margin—rather than aspect ratio alone. Second, replace the remaining HiGHS face selectors with the maintained basis and bounded native repairs. Third, close the three-program coverage gap without slowing the cheap branch. The walker now covers part of the left-hand synthetic gap, and we measure the remaining benchmark deficit under one accuracy standard.

5 Related work and a provisional Pinar check

The projection path itself is classical. Mangasarian and Meyer established finite exactness for nonlinear LP perturbations in 1979, and Mangasarian later characterized least-norm LP solutions by projection [Mangasarian and Meyer, 1979, Mangasarian, 1984]. Madsen, Nielsen, and Pinar developed finite continuation methods on piecewise-linear paths [Madsen et al., 1996, Pınar, 1996]. Most directly, Pinar's 1997 algorithm follows a quadratic-penalty path with predictor steps, modified-Newton correction, retained factor updates, and iterative refinement [Pınar, 1997]. This work predates Mangasarian's 2002 Newton report, published in 2004 [Mangasarian, 2004, Kline and Fung, 2023].

After dualization and the change of parameter $t = 1/\tau$, Pinar's penalty optimizer is exactly $P_D(tb)$, the path used here. The distinction is algorithmic. Pinar evaluates selected penalty values with predictor and corrector steps; t-walker attempts to visit the next certified breakpoint. General parametric-QP active-set methods already supply the larger setting for piecewise-affine optimizer maps, ratio events, dependent-constraint exchanges, and factor updates [Best, 1996, Potschka et al., 2010, Ferreau et al., 2014]. Bartels also used reduced-gradient basis exchanges in a penalty LP method [Bartels, 1980]. We therefore do not claim a new path or homotopy principle.

The exact algebra is short, and we give it against Pinar's own equations rather than a paraphrase. We rename two of his symbols to keep our own free: his right-hand side becomes a and his penalty parameter τ, where he writes b and t. He states the standard-form pair

$$\text{(P)}\ \min_z c^\mathsf{T} z \text{ s.t. } Az = a,\ z \geq 0, \qquad \text{(D)}\ \max_y -a^\mathsf{T} y \text{ s.t. } A^\mathsf{T} y + c \geq 0,$$

penalizes the constraints of (D), and studies the *unconstrained* problem

$$\text{(CD)} \quad \min_y\ H(y,\tau) = \tau\, a^\mathsf{T} y + \tfrac{1}{2}\|(A^\mathsf{T} y + c)_-\|^2$$

for decreasing $\tau > 0$; this is his equations (4)–(6), with W the diagonal selector of violated rows. He then records, on p. 623, the dual of (CD) as the constrained regularized program

$$\text{(PB)} \quad \min_z\ c^\mathsf{T} z + \frac{\tau}{2}\|z\|^2 \text{ s.t. } Az = a,\ z \geq 0.$$

Put $A = B^\mathsf{T}$, $a = d$, $c = -b$, so that his (P) is our dual and his (D) is our primal. Either problem now delivers our path. Dividing H by τ and writing $t = 1/\tau$ turns (CD) into

$$\min_x\ d^\mathsf{T} x + \frac{t}{2}\|(b - Bx)_+\|^2,$$

Mangasarian's penalty program for our primal. And (PB) reads $\min\{-b^\mathsf{T} y + (\tau/2)\|y\|^2 : y \in D\}$, whose optimizer is

$$\arg\min_{y \in D} \tfrac{1}{2}\|y - b/\tau\|^2 = P_D(b/\tau) = P_D(tb) = y(t).$$

His perturbation result identifies that point, for τ small, with the least-2-norm solution of his (P) — our dual. His parameter decreases where ours increases. Recent work sharpens finite-exactness thresholds and shows that regularization paths need not obey simple support-monotonicity rules [González-Sanz and Nutz, 2025, González-Sanz et al., 2025]. The supportable contribution here is narrower: a sparse direct breakpoint implementation of this classical path, original-data certification and rank-deficient-face repairs, and an empirical demonstration that face following complements a Mangasarian-style Newton solver in the measured low-ratio regime.

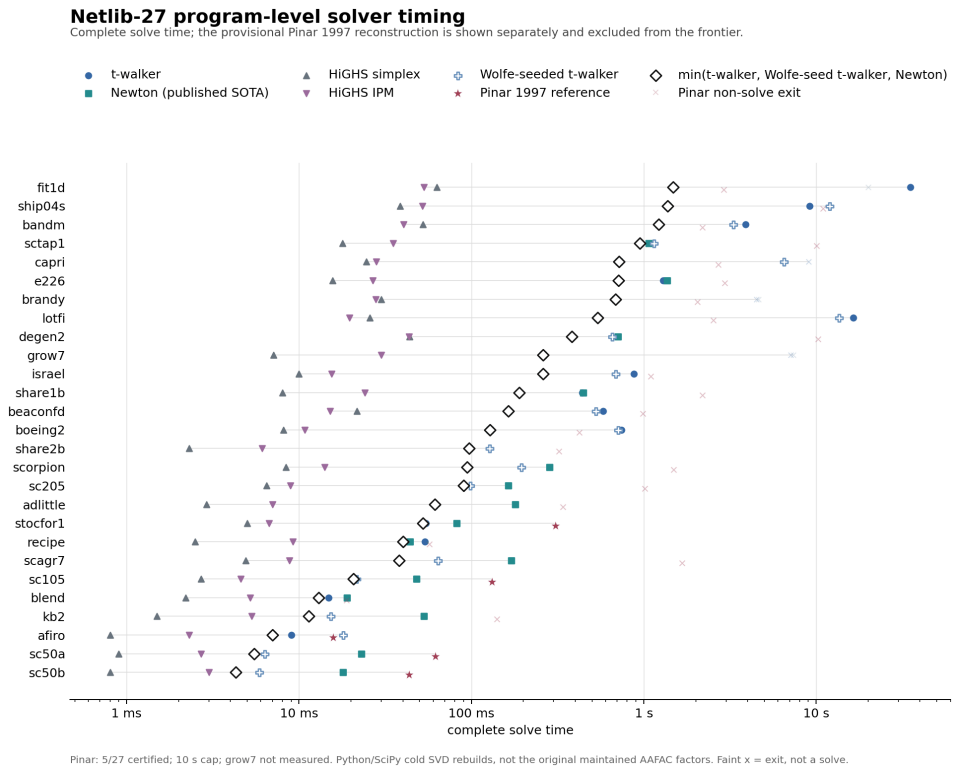

Figure 3: Complete solve time on Netlib-27. Stars mark the five certified results from the provisional Pinar reconstruction; faint crosses mark measured exits, not solves. Pinar is excluded from the black Mangasarian frontier. The other solver values are frozen cached records.

We built a correctness-first reconstruction of Pinar's algorithm and ran it on the same canonical Netlib panel with one thread and a 10-second cap. It called neither t-walker nor an LP solver. It certified `afiro`, `sc50b`, `sc50a`, `sc105`, and `stocfor1`; 18 models stopped at a numerical gate, three reached the time limit, and `grow7` was not measured because its compact panel fixture could not be exported—the shipped fixture manifest records `initialization did not produce an accepted face`—although the canonical MPS file is present. The five complete Python/SciPy times ranged from 15.7 to 306.9 ms. Two identical repetitions returned the same status on every model.

These timings are directional, not a reproduction of Pinar's Fortran costs. For smaller models, our code rebuilds a cold rank-revealing SVD at every Newton and path solve; larger models use sparse LSMR. Pinar maintained and updated AAFAC factors, and his paper reports successful runs on its ten-model panel. One possible reading of our failures is that a geometric predictor and corrector may still need an explicit combinatorial basis/status policy at degenerate breakpoints. That is a hunch to test, not a conclusion from 5 of 27.

6 Sources and protocol

The TeX source and BibTeX file are the canonical and only inputs to this PDF. Current reproductions are, with the first four under `experiments/`:

- timings: `render_solver_timing_charts.py`;
- accuracy: `bench_common_accuracy.py` and `render_common_accuracy.py`;

- provisional Pinar panel: `pinar1997/run_netlib_panel.py` and `pinar1997/render_provisional_netlib.py`;
- Netlib walker panel: `bench_twalker_netlib_panel.py`; and
- this PDF: `paper/build_paper.sh`.

One thread; HiGHS presolve off; IPM uses both crossover settings. We report five interleaved runs for raw synthetic values and one deterministic warm-point run per problem for crossover accuracy. The Netlib panel combines frozen records, so read its times at order-of-magnitude precision.

Jeff Kline directed the mathematical and computational work. AI systems helped draft code, run bounded experiments, audit claims, and edit this prototype. They are neither authors nor referees, and their agreement with each other is evidence about the checking process, not a certificate of correctness. The repository identifies the measured executable, preserves the compact raw records, and documents which aggregates it re-derives and which it does not; `VERIFICATION.md` names the reproduction gaps and `CORRECTIONS.md` records material corrections. The literature review was bounded rather than exhaustive; the strongest novelty language should remain provisional until an optimization specialist reviews the equation map and bibliography.

References

Olvi L. Mangasarian and Robert R. Meyer. Nonlinear perturbation of linear programs. *SIAM Journal on Control and Optimization*, 17(6):745–752, 1979. doi: 10.1137/0317052.

Olvi L. Mangasarian. Normal solutions of linear programs. In *Mathematical Programming Study 22*, pages 206–216. Springer, 1984. doi: 10.1007/BFb0121017.

Olvi L. Mangasarian. A newton method for linear programming. *Journal of Optimization Theory and Applications*, 121(1):1–18, 2004. doi: 10.1023/B:JOTA.0000026128.34294.77. First circulated as technical report DMI 02-02 in 2002.

Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018. doi: 10.1007/s12532-017-0130-5.

Kaj Madsen, Hans Bruun Nielsen, and Mustafa C. Pınar. A new finite continuation algorithm for linear programming. *SIAM Journal on Optimization*, 6(3):600–616, 1996. doi: 10.1137/S1052623493258556.

Mustafa C. Pınar. Linear programming via a quadratic penalty function. *Mathematical Methods of Operations Research*, 44(3):345–370, 1996. doi: 10.1007/BF01193936.

Mustafa C. Pınar. Piecewise-linear pathways to the optimal solution set in linear programming. *Journal of Optimization Theory and Applications*, 93(3):619–634, 1997. doi: 10.1023/A:1022651331550. URL `https://repository.bilkent.edu.tr/items/f49199ba-5005-4867-9f87-f9a084fd5b69`.

Jeff Kline and Glenn Fung. Linear programming with nonparametric penalty programs and iterated thresholding. *Optimization Methods and Software*, 38(1):107–127, 2023. doi: 10.1080/10556788.2022.2117356.

Michael J. Best. An algorithm for the solution of the parametric quadratic programming problem. In *Applied Mathematics and Parallel Computing*, pages 57–76. Physica-Verlag, 1996. doi: 10.1007/978-3-642-99789-1_5.

Andreas Potschka, Christian Kirches, Hans Georg Bock, and Johannes P. Schlöder. Reliable solution of convex quadratic programs with parametric active set methods. Technical report, Optimization Online, 2010. URL `https://optimization-online.org/2010/11/2828/`.

Hans Joachim Ferreau, Christian Kirches, Andreas Potschka, Hans Georg Bock, and Moritz Diehl. qpOASES: A parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6:327–363, 2014. doi: 10.1007/s12532-014-0071-1.

Richard H. Bartels. A penalty linear programming method using reduced-gradient basis-exchange techniques. *Linear Algebra and its Applications*, 29:17–32, 1980. doi: 10.1016/0024-3795(80)90227-X.

Alberto González-Sanz and Marcel Nutz. Quantitative convergence of quadratically regularized linear programs. *Applied Mathematics & Optimization*, 91(3), 2025. doi: 10.1007/s00245-025-10267-1.

Alberto González-Sanz, Marcel Nutz, and Andrés Riveros Valdevenito. Monotonicity in quadratically regularized linear programs. *SIAM Journal on Optimization*, 35(2):1419–1437, 2025. doi: 10.1137/24M1685171.